APOSTILA — ATAQUE DE INIMIGO

Unity 6 • Animator • C# • Animation Events

Implementando um ataque que para o inimigo,
espera o momento correto da animação e aplica dano.

Material de apoio para aula prática

1. Objetivo da aula

Nesta prática vamos adicionar um ataque ao inimigo usando o Animator e um script C#. O inimigo deverá permanecer no alcance do jogador por 0,5 segundo antes de iniciar o golpe. Quando o ataque começar, o inimigo para, olha para o jogador e executa a animação. O dano não acontece durante toda a animação: ele acontece somente entre dois Animation Events.

Ao final, o fluxo será:

- Jogador entra no alcance do inimigo.
- O inimigo permanece no alcance por 0,5 segundo.
- O inimigo para de andar e olha para o jogador.
- O Animator recebe o Trigger Attack.
- O Animation Event Start abre a janela de dano.
- O script verifica a colisão com o jogador.
- O Animation Event End fecha a janela de dano.
- O inimigo volta a se mover.

2. Preparação do inimigo

O GameObject do inimigo deve possuir, pelo menos, os seguintes componentes:

- Animator
- NavMeshAgent
- EnemyVisionSensor
- Patrol
- Attack

O EnemyVisionSensor utilizado na aula já possui a referência ao jogador e a propriedade PlayerSeen. O Patrol utiliza NavMeshAgent e Animator para controlar o movimento e a animação.

Estrutura sugerida:

```
Enemy
├── Animator
├── NavMeshAgent
├── EnemyVisionSensor
├── Patrol
├── Attack
└── AttackPoint
```

3. Criando o AttackPoint

Crie um GameObject vazio como filho do inimigo e dê a ele o nome AttackPoint. Posicione esse objeto na região em que o golpe deverá atingir, por exemplo, à frente da mão, arma ou corpo do inimigo.

No Inspector do componente Attack:

- Attack Range: distância máxima para iniciar o ataque.
- Time To Attack: tempo que o jogador precisa permanecer dentro do alcance. Use 0,5.
- Attack Point: arraste o objeto AttackPoint para este campo.

- Attack Radius: raio da área de colisão do golpe.
- Damage: quantidade de dano enviada ao jogador.

DICA
Ative Gizmos e selecione o inimigo para visualizar a esfera vermelha do AttackPoint. A esfera representa a área usada pelo OverlapSphere.

4. Configurando o Animator

No Animator, crie um parâmetro do tipo Trigger chamado exatamente:

```
Attack
```

Crie ou utilize o estado de animação de ataque e faça a transição para ele usando o Trigger Attack.

Exemplo:

```
idle / walking
      |
      |  Attack (Trigger)
      v
     attk
      |
      |  saída da animação
      v
idle / walking
```

ATENÇÃO
O nome do Trigger no Animator precisa ser exatamente "Attack", pois o script usa animator.SetTrigger("Attack").

5. Adicionando os Animation Events

Os Animation Events são usados para sincronizar o código com um frame específico da animação. Neste ataque teremos dois eventos: Start e End.

5.1 Evento Start

Abra a animação de ataque na janela Animation. Escolha o frame em que a arma ou mão realmente deve causar dano. Nesse frame, adicione um Animation Event e selecione a função pública:

```
Start()
```

A partir desse evento, o script começa a verificar se o jogador está dentro da área de ataque.

5.2 Evento End

Escolha o frame em que a janela de dano termina. Adicione outro Animation Event e selecione:

```
End()
```

Depois de End(), o ataque deixa de verificar colisões e o inimigo pode voltar a se mover.

Visualmente:

```
Animação de ataque
|-----------------------------------------------|
                 ^                    ^
               Start                End
                 |                    |
          começa o dano          termina o dano
```

IMPORTANTE
Não coloque Start no primeiro frame simplesmente por colocar. O evento deve ficar no momento visual em que o golpe atinge o alvo.

6. Entendendo o código Attack.cs

6.1 Variáveis principais

```
public float attackRange = 2f;
public float timeToAttack = 0.5f;
public float damage = 10f;

public Transform attackPoint;
public float attackRadius = 1f;
```

attackRange define até onde o jogador pode estar para o ataque ser iniciado. timeToAttack define os 0,5 segundo de espera. attackPoint define o centro da área de golpe e attackRadius define o tamanho dessa área.

6.2 Encontrando o jogador

```
GameObject obj = GameObject.FindGameObjectWithTag("Player");

if (obj != null)
    player = obj.transform;
```

O script procura um GameObject com a Tag Player e guarda seu Transform. Por isso, o personagem do jogador deve estar com a Tag Player.

6.3 Esperando 0,5 segundo

```
if (distance <= attackRange)
{
    timeInRange += Time.deltaTime;

    if (timeInRange >= timeToAttack)
    {
        StartAttack();
```

```
    }
}
else
{
    timeInRange = 0f;
}
```

Enquanto o jogador estiver dentro do alcance, timeInRange aumenta. Quando chegar a 0,5 segundo, StartAttack() é chamado. Se o jogador sair do alcance antes disso, o contador é zerado.

6.4 Parando e iniciando o ataque

```
attacking = true;
agent.isStopped = true;

Vector3 direction = player.position - transform.position;
direction.y = 0f;

if (direction != Vector3.zero)
    transform.rotation = Quaternion.LookRotation(direction);

animator.SetTrigger("Attack");
```

O NavMeshAgent é parado, o inimigo gira horizontalmente para o jogador e o Trigger Attack é enviado ao Animator.

6.5 O evento Start

```
public void Start()
{
    checkingHit = true;
    alreadyHit = false;
}
```

Start é público para poder ser chamado diretamente por um Animation Event. Ele abre a janela em que o ataque pode acertar o jogador.

6.6 Detectando o golpe

```
Collider[] hits = Physics.OverlapSphere(
    attackPoint.position,
    attackRadius
);
```

OverlapSphere cria uma área esférica de detecção. Se um Collider do jogador estiver dentro dela, o código considera que o golpe acertou.

6.7 Evitando vários danos no mesmo golpe

```
if (alreadyHit)
    return;
```

A variável alreadyHit impede que o jogador receba dano repetidamente enquanto a janela Start → End estiver aberta.

6.8 Aplicando o dano

```
hit.SendMessage(
    "TakeDamage",
    damage,
    SendMessageOptions.DontRequireReceiver
);
```

O script envia a mensagem TakeDamage para o objeto atingido. O script do jogador pode implementar um método público TakeDamage para receber esse valor.

6.9 O evento End

```
public void End()
{
    checkingHit = false;
    attacking = false;

    agent.isStopped = false;
}
```

End fecha a janela de dano, permite que um novo ataque seja iniciado e libera o NavMeshAgent para voltar a se mover.

7. Fluxo completo do sistema

```
Jogador entra no alcance
        ↓
Contador começa
        ↓
Permanece 0,5s no alcance?
        ↓ SIM
Inicia ataque
        ↓
NavMeshAgent para
        ↓
Animator → Attack
        ↓
Animation Event → Start
        ↓
OverlapSphere verifica o jogador
        ↓
Acertou? → TakeDamage()
        ↓
Animation Event → End
        ↓
Fecha janela de dano
        ↓
NavMeshAgent volta a andar
```

8. Checklist para testar

☐ O inimigo possui Animator.

☐ Existe um Trigger chamado Attack.

☐ A animação de ataque está conectada ao estado attk.

☐ O GameObject do jogador possui a Tag Player.

☐ O inimigo possui NavMeshAgent.

☐ O script Attack está no inimigo.

☐ O AttackPoint foi criado e atribuído no Inspector.

☐ O Attack Range está ajustado.

☐ Time To Attack está em 0,5.

☐ A animação possui o evento Start no frame do impacto.

☐ A animação possui o evento End no final da janela de impacto.

☐ O jogador possui um método TakeDamage para receber o dano.

9. Exercício para o aluno

Configure o inimigo para atacar o jogador. O inimigo deve esperar 0,5 segundo dentro do alcance, parar durante o ataque, executar a animação e aplicar dano somente na janela delimitada pelos eventos Start e End.

Depois, teste e ajuste:

- Attack Range.
- Attack Radius.
- Damage.
- Posição do AttackPoint.
- Frame do Start.
- Frame do End.

10. Problemas comuns

O ataque não começa: Verifique se o parâmetro Attack é um Trigger e se o nome está escrito exatamente igual no Animator e no script.

O jogador não recebe dano: Verifique a Tag Player, a posição do AttackPoint, o Attack Radius e se o jogador possui um método TakeDamage.

O dano acontece antes do impacto: Mova o Animation Event Start para o frame correto da animação.

O dano acontece várias vezes: Verifique se o código está mantendo alreadyHit como true após o primeiro acerto.

O inimigo não volta a andar: Verifique se o Animation Event End está configurado e chamando a função End().

11. Código completo — Attack.cs

```
using UnityEngine;
using UnityEngine.AI;

public class Attack : MonoBehaviour
{
    [Header("Ataque")]
    public float attackRange = 2f;
    public float timeToAttack = 0.5f;
    public float damage = 10f;

    [Header("Colisão do ataque")]
    public Transform attackPoint;
    public float attackRadius = 1f;

    private Transform player;
    private NavMeshAgent agent;
    private Animator animator;

    private float timeInRange = 0f;
    private bool attacking = false;
    private bool checkingHit = false;
    private bool alreadyHit = false;

    void Start()
    {
        agent = GetComponent<NavMeshAgent>();
        animator = GetComponent<Animator>();

        GameObject obj = GameObject.FindGameObjectWithTag("Player");

        if (obj != null)
            player = obj.transform;
    }

    void Update()
    {
        if (player == null)
            return;

        float distance = Vector3.Distance(transform.position, player.position);

        if (attacking)
        {
            if (checkingHit)
                CheckHit();

            return;
        }

        if (distance <= attackRange)
        {
            timeInRange += Time.deltaTime;

            if (timeInRange >= timeToAttack)
            {
                StartAttack();
            }
        }
```

```
        else
        {
            timeInRange = 0f;
        }
    }

    void StartAttack()
    {
        attacking = true;
        timeInRange = 0f;

        agent.isStopped = true;

        Vector3 direction = player.position - transform.position;
        direction.y = 0f;

        if (direction != Vector3.zero)
            transform.rotation = Quaternion.LookRotation(direction);

        animator.SetTrigger("Attack");
    }

    // Animation Event:
    // Coloque este evento no frame em que o golpe deve causar dano.
    public void Start()
    {
        checkingHit = true;
        alreadyHit = false;
    }

    // Animation Event:
    // Coloque este evento no final da janela de dano.
    public void End()
    {
        checkingHit = false;
        attacking = false;

        agent.isStopped = false;
    }

    void CheckHit()
    {
        if (alreadyHit || attackPoint == null)
            return;

        Collider[] hits = Physics.OverlapSphere(
            attackPoint.position,
            attackRadius
        );

        foreach (Collider hit in hits)
        {
            if (hit.CompareTag("Player"))
            {
                alreadyHit = true;

                hit.SendMessage(
                    "TakeDamage",
                    damage,
                    SendMessageOptions.DontRequireReceiver
```

```
                );

                Debug.Log("Player atingido!");
                break;
            }
        }
    }

    void OnDrawGizmosSelected()
    {
        if (attackPoint == null)
            return;

        Gizmos.color = Color.red;
        Gizmos.DrawWireSphere(
            attackPoint.position,
            attackRadius
        );
    }
}
```